



PRACTICA 2

**Node.js + Express y Thunder Client +
(sql puro u orm)**

NOMBRE:

Medina Pajsi Katherin

C.I.:

9928455

FECHA:

02/06/2026

Ejercicios

Los siguientes ejercicios pruebe el endpoint usando Thunder Client.

1. Crea un endpoint POST /categorias que permita registrar una nueva categoría enviando nombre y descripción en el body de la petición.

```
app.post("/", async (req, res) => {
  try {
    const { nombre, descripcion, createdAt, updatedAt } = req.body;

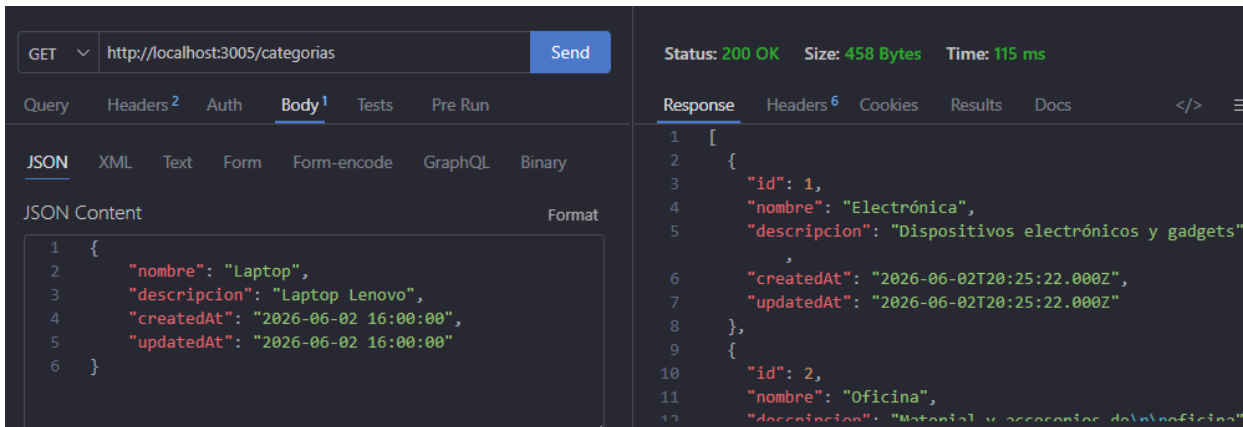
    const [resultado] = await pool.query(
      "INSERT INTO categorias (nombre, descripcion, createdAt, updatedAt) VALUES (?, ?, ?, ?)",
      [nombre, descripcion, createdAt, updatedAt],
    );

    res.status(201).json(resultado);
  } catch (error) {
    console.error(error);
    res.status(500).json({
      mensaje: "Error al insertar el producto",
    });
  }
});
```

2. Crea un endpoint GET /categorias que devuelva todas las categorías registradas en la base de datos.

```
app.get("/categorias", async (req, res) => {
  try {
    const [categorias] = await pool.query("SELECT * FROM categoria");

    res.json(categorias);
  } catch (error) {
    console.log(error);
    res.status(500).json({
      mensaje: "Error al obtener categorías",
    });
  }
});
```



3. Crea un endpoint GET `/categorias/:id` que devuelva la categoría con el ID especificado y además, incluya todos los productos que pertenecen a esa categoría.

-creamos la tabla productos

```
CREATE TABLE producto(
  id INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100),
  descripcion VARCHAR(255),
  categoriaId INT,
  createdAt DATETIME,
  updatedAt DATETIME,
  FOREIGN KEY(categoriaId) REFERENCES categoria(id)
);
```

-la solución:

```
app.get("/categorias/:id", async (req, res) => {
  try {
    const { id } = req.params;

    const [categoria] = await pool.query("SELECT * FROM categorias WHERE id=?", [
      id,
    ]);

    if (categoria.length === 0) {
      return res.status(404).json({
        mensaje: "Categoría no encontrada",
      });
    }

    const [productos] = await pool.query(
```

```

        "SELECT * FROM producto WHERE cid=?",
        [id],
    );

    res.json({
        categoria: categoria[0],
        productos,
    });
} catch (error) {
    console.log(error);
    res.status(500).json({
        mensaje: "Error al obtener categoría",
    });
}
});

```

The screenshot shows a REST client interface. The top bar indicates a GET request to `http://localhost:3005/categorias/2` with a status of 200 OK, size of 528 Bytes, and time of 90 ms. The 'Body' tab is selected, showing the JSON response. The response is a JSON array with two objects. The first object represents a category with 'id' 3, 'nombre' 'Impresora Epson', 'descripcion' 'Impresora multifuncional Epson L3250', 'cid' 2, 'createdAt' '2026-06-02T21:29:44.000Z', and 'updatedAt' '2026-06-02T21:29:44.000Z'. The second object represents a category with 'id' 4, 'nombre' 'Silla Ejecutiva', and 'descripcion' 'Silla ergonomica para oficina'.

Tab	Content
Query	<code>http://localhost:3005/categorias/2</code>
Headers	2
Auth	
Body	1
Tests	
Pre Run	
Response	6
Headers	6
Cookies	
Results	
Docs	

JSON Content

```

1 {
2   "nombre": "Laptop",
3   "descripcion": "Laptop Lenovo",
4   "createdAt": "2026-06-02 16:00:00",
5   "updatedAt": "2026-06-02 16:00:00"
6 }

```

Response

```

11 "id": 3,
12 "nombre": "Impresora Epson",
13 "descripcion": "Impresora multifuncional Epson
14   L3250",
15 "cid": 2,
16 "createdAt": "2026-06-02T21:29:44.000Z",
17 "updatedAt": "2026-06-02T21:29:44.000Z"
18 },
19 {
20   "id": 4,
21   "nombre": "Silla Ejecutiva",
22   "descripcion": "Silla ergonomica para oficina",
23   "createdAt": "2026-06-02T21:29:44.000Z",
24   "updatedAt": "2026-06-02T21:29:44.000Z"
25 }

```

4. Crea un endpoint PATCH /categorias/:id que permita actualizar todos los datos de la categoría con el ID especificado.

```

app.patch("/categorias/:id", async (req, res) => {
    try {
        const { id } = req.params;
        const { nombre, descripcion, updatedAt } = req.body;

        const [resultado] = await pool.query(
            `UPDATE categorias
            SET nombre=?, descripcion=?, updatedAt=?
            WHERE id=?`,
            [nombre, descripcion, updatedAt, id],
        );
    } catch (error) {
        console.log(error);
        res.status(500).json({
            mensaje: "Error al actualizar categoría",
        });
    }
});

```

```

    );

    if (resultado.affectedRows === 0) {
        return res.status(404).json({
            mensaje: "Categoría no encontrada",
        });
    }

    res.json({
        mensaje: "Categoría actualizada",
    });
} catch (error) {
    console.log(error);
    res.status(500).json({
        mensaje: "Error al actualizar categoría",
    });
}
});

```

5. Crea un endpoint DELETE /categorias/:id que elimine la categoría indicada y, al mismo tiempo, elimine automáticamente todos los productos que pertenecen a esa categoría.

```

app.delete("/categorias/:id", async (req, res) => {
    try {
        const { id } = req.params;

        const [resultado] = await pool.query("DELETE FROM categorias WHERE
id=?", [
            id,
        ]);

        if (resultado.affectedRows === 0) {
            return res.status(404).json({
                mensaje: "Categoría no encontrada",
            });
        }

        res.json({
            mensaje: "Categoría eliminada",
        });
    }
});

```

```

    });
  } catch (error) {
    console.log(error);
    res.status(500).json({
      mensaje: "Error al eliminar categoría",
    });
  }
});

const PUERTO = 3005;
app.listen(PUERTO, () => {
  console.log(`Servidor backend en http://localhost:${PUERTO}`);
});

```

The screenshot shows a REST client interface with a DELETE request to `http://localhost:3005/categorias/3`. The response is a 200 OK status with a JSON body containing a success message.

Method	URL	Status	Size	Time
DELETE	http://localhost:3005/categorias/3	200 OK	34 Bytes	90 ms

Tab	Content
Body	<pre>{ "nombre": "Laptop", "descripcion": "Laptop Lenovo", "createdAt": "2026-06-02 16:00:00", "updatedAt": "2026-06-02 16:00:00" }</pre>
Response	<pre>{ "mensaje": "Categoría eliminada" }</pre>